

Transformer-Driven Cost Matrix Adaptation in Model Predictive Control of DC Motors

Silvia-Teodora Porcarasu

*Department of Automatic Control and Applied Informatics
Gheorghe Asachi Technical University of Iasi
Iasi, Romania
silvia-teodora.porcarasu@student.tuiasi.ro*

Denisa-Gabriela Musteata

*Department of Automatic Control and Applied Informatics
Gheorghe Asachi Technical University of Iasi
Iasi, Romania
denisa-gabriela.musteata@student.tuiasi.ro*

Constantin-Florin Caruntu

*Department of Automatic Control and Applied Informatics
Gheorghe Asachi Technical University of Iasi
Iasi, Romania
caruntuc@ac.tuiasi.ro*

Abstract—In MPC-based DC motor control, the cost matrices play a key role in shaping closed-loop behavior, but their tuning is often scenario-dependent and difficult to generalize across operating conditions. This paper addresses this limitation through a Transformer-driven framework for data-driven cost matrix adaptation. The objective is to reduce the need for manual controller tuning by learning the relationship between motor operating conditions and suitable MPC cost function parameters. A dataset is generated using real-fitted DC motor parameters, simulated reference and disturbance profiles, and offline MPC-based optimization. The architecture employs sequential and static features, a Transformer encoder with attention pooling, and a mixed loss function to learn the mapping between operating scenarios and the optimized MPC weighting matrices P , Q , and R . By enabling data-driven adaptation of the MPC cost matrices to varying operating conditions, the method aims to improve controller flexibility while maintaining control performance. The model is evaluated using absolute-scale and logarithmic-scale metrics, including global R^2 scores, relative errors, and log-domain MAE/RMSE. The results demonstrate that the approach can predict suitable MPC cost parameters across a wide range of operating scenarios, highlighting the potential of Transformer-based models for adaptive predictive control of DC motors.

Index Terms—Model Predictive Control, DC motor, Transformer, cost matrix adaptation, data-driven control, neural networks

I. INTRODUCTION

Model Predictive Control (MPC) is widely used for constrained dynamical systems due to its ability to explicitly handle state and input constraints while optimizing future system behavior over a prediction horizon [1], [2]. Its performance depends on the prediction horizon, sampling time, and on the parameters defining the cost function. Selecting these parameters is often application-dependent and typically requires expert knowledge or computationally expensive optimization [3]. As operating conditions change, fixed controller

settings may no longer provide the desired balance between tracking performance, control effort, and robustness.

Data-driven methods have demonstrated the potential to learn relationships between system characteristics and appropriate controller configurations, offering a computationally efficient alternative to repeated online tuning [4], [5]. In particular, learning-based predictive control approaches have shown that machine learning techniques can support controller design and adaptation while reducing the dependence on repeated optimization procedures [6], [7]. These developments motivate the use of data-driven models capable of predicting suitable controller parameters directly from operating conditions.

Recent surveys on data-driven and learning-based MPC highlight two main research directions. The first one focuses on adapting or identifying prediction models from data while preserving the standard online MPC optimization structure. The second one uses machine learning models to approximate parts of the MPC pipeline, such as the control policy, the optimizer, or the tuning parameters, in order to reduce online computational effort. Neural approximations of MPC have been investigated as a way to accelerate control computation, but their practical use still depends on generalization capability, closed-loop reliability, and the preservation of meaningful control behavior across operating conditions [8], [9].

Another relevant research direction concerns the learning of MPC cost functions or controller parameters from closed-loop data. Instead of replacing the predictive controller entirely, these methods retain the MPC structure and use learning only to adapt selected components of the optimization problem. This is particularly attractive when constraints and model-based prediction must remain explicit, but manual tuning of the cost function is time-consuming or scenario-dependent. Recent work on neural-network-supported MPC has also emphasized the importance of safety and stability when learning controller parameters, especially when the learned components influence the closed-loop response [10].

Part of this research is supported by the project “Romanian Hub for Artificial Intelligence - HRIA”, Smart Growth, Digitization and Financial Instruments Program, 2021-2027, MySMIS no. 334906.

In modern machine learning architectures, Transformer models have attracted considerable attention in time-series analysis due to their ability to capture long-range temporal dependencies through self-attention mechanisms [11], [12]. Recent studies have investigated Transformer-based approaches for MPC acceleration and trajectory optimization [13], [14], while more recent work has explored Transformer-based formulations for explicit and adaptive MPC implementations [15]. In the considered problem, the input contains temporally ordered reference, motor-response, control, and disturbance signals. The Transformer was therefore selected to model dependencies across the observation window through self-attention, while a separate encoder processes the static motor and scenario descriptors.

Motivated by the need to reduce repeated MPC tuning, this work proposes a Transformer-based method for adapting the cost matrices P , Q , and R to different DC motor operating conditions. The method retains the constrained MPC formulation and learns only the cost parameters from temporal operating data and static motor descriptors. Training data are obtained from experimentally fitted motor models, simulated scenarios, and offline MPC tuning. The main contributions are:

- a data-driven method for scenario-dependent adaptation of P , Q , and R without replacing the MPC controller;
- a dataset-generation procedure combining experimentally fitted motor models, simulated reference and disturbance profiles, and offline MPC tuning;
- a Transformer regressor that fuses temporal and static features to predict the log-diagonal MPC weights;
- a comparison of temporal encoders, pooling strategies, dataset sizes, and loss functions;
- parameter-level and closed-loop evaluations covering regression accuracy, tracking performance, and stability.

The remainder of this paper is organized as follows. Section II presents the DC motor model and the MPC formulation. Section III describes the dataset generation process and the optimization procedure used to obtain the target controller parameters. Section IV presents the Transformer framework and training methodology. Section V presents the experimental setup and evaluation results. Finally, Section VI presents the conclusions and outlines future research directions.

II. PLANT MODELING AND CONTROLLER DESIGN

The considered plant is an armature-controlled DC motor regulated by a finite-horizon MPC strategy. MPC is suitable for this class of systems because it allows tracking objectives, actuator limits, and control-effort penalization to be handled explicitly within a constrained optimization problem [16]. In this work, the same discrete-time model is used for closed-loop simulation, offline controller tuning, and dataset generation for the learning-based adaptation stage.

The motor dynamics are described by the standard electromechanical equations commonly used for DC motor modeling and identification [17], [18]. The discrete-time state vector is defined as

$$x_k = [i_{a,k} \quad \omega_k]^T, \quad (1)$$

where k is the discrete-time index, $i_{a,k}$ is the armature current, and ω_k is the angular velocity. After zero-order-hold discretization with sampling time $T_s = 0.02$, the motor model x_{k+1} is written as

$$x_{k+1} = A_d x_k + B_d u_k + E_d d_k, \quad (2)$$

$$y_k = C x_k, \quad (3)$$

where x_{k+1} is the predicted state at the next sampling instant, u_k is the armature voltage applied to the motor, d_k is the load disturbance, and y_k is the measured output. The matrices A_d , B_d , and E_d describe the discrete-time effects of the current state, control input, and load disturbance, respectively. Since the controlled output is the angular velocity, the output matrix is selected as $C = [0 \ 1]$, which extracts ω_k from the state vector. At each sampling instant, the MPC controller uses the current state and the discrete-time prediction model to compute a sequence of future voltage inputs. The optimization is performed over a prediction horizon N , where N denotes the number of future sampling instants considered by the controller. Only the first element of the optimized control sequence is applied to the motor, and the same procedure is repeated at the next sampling instant according to the receding-horizon principle. The predicted tracking error at step $k+j$ is defined as $e_{k+j} = x_{k+j} - x_{\text{ref}}$, where $j = 0, \dots, N$ is the prediction-step index, x_{k+j} is the predicted state at step $k+j$, and x_{ref} is the reference state associated with the desired angular velocity reference. For the prediction horizon N , the quadratic MPC objective is

$$J = e_{k+N}^T P e_{k+N} + \sum_{j=0}^{N-1} (e_{k+j}^T Q e_{k+j} + u_{k+j}^T R u_{k+j}), \quad (4)$$

where J the total objective is minimized by the controller. The first term, $e_{k+N}^T P e_{k+N}$, penalizes the terminal tracking error at the end of the prediction horizon. The summation term accumulates the penalties over all intermediate prediction steps: $e_{k+j}^T Q e_{k+j}$ penalizes the predicted tracking error, while $u_{k+j}^T R u_{k+j}$ penalizes the predicted control effort. Thus, Q , R , and P are weighting terms introduced in the MPC objective: Q weights state-tracking performance, R weights the control effort, and P weights the terminal tracking error.

The control input sequence is obtained by solving the following constrained optimization problem over the prediction horizon of N steps:

$$\begin{aligned} \min_u \quad & J \\ \text{s.t.} \quad & x_{k+j+1} = A_d x_{k+j} + B_d u_{k+j} + E_d d_{k+j}, \\ & u_{\min} \leq u_{k+j} \leq u_{\max}, \end{aligned} \quad (5)$$

for $j = 0, \dots, N-1$, where $u = \{u_k, \dots, u_{k+N-1}\}$ is the sequence of voltage inputs to be optimized. The first constraint ensures the discrete-time motor dynamics at each prediction step, ensuring that the predicted states evolve consistently with the motor model. The second constraint limits the applied voltage to the admissible bounds $u_{\min}$ and $u_{\max}$, representing

the physical voltage limits of the motor drive. Only the first element u_k of the optimized sequence is applied to the motor, after which the optimization is repeated at the next time step following the receding horizon principle.

The selection of P , Q , and R directly affects the closed-loop behavior of the MPC controller. A larger tracking-error weight generally prioritizes reference following, whereas a larger control-effort weight leads to smoother and less aggressive voltage commands. Since the appropriate compromise depends on the motor parameters, the reference trajectory, and the disturbance profile, manual tuning of these weighting terms can require repeated simulations or optimization procedures [6].

In this work, the tuning of the MPC objective is formulated as a supervised learning problem. Offline MPC-based optimization is first used to determine suitable values of the weighting terms P , Q , and R for different operating scenarios. These optimized quantities are then used as learning targets for the Transformer-based model, which receives static motor-related features and sequential operating-condition features as inputs and predicts the weighting terms used to configure the MPC controller.

For stability assessment, the predicted weighting terms are also analyzed through the unconstrained feedback law associated with the quadratic MPC formulation. When P is chosen consistently with the discrete-time algebraic Riccati equation corresponding to a given pair Q and R , the terminal term $e_{k+N}^T P e_{k+N}$ has the usual Lyapunov interpretation for the unconstrained tracking-error dynamics [19]. The associated feedback gain is $K = (R + B_d^T P B_d)^{-1} B_d^T P A_d$, where K maps the current state or tracking error to the corresponding linear feedback control action in the unconstrained case. This gain defines the closed-loop state-transition matrix as $A_{cl} = A_d - B_d K$. The matrix A_{cl} describes how the state or tracking error evolves from one sampling instant to the next when the feedback law induced by the quadratic formulation is applied. Its eigenvalues are used as a stability diagnostic: if all eigenvalues lie inside the unit circle, the corresponding unconstrained discrete-time closed-loop system is asymptotically stable. Therefore, in the evaluation stage, the spectral properties A_{cl} are used to verify whether the Transformer-predicted weighting terms lead to stable unconstrained closed-loop behavior. All 300 validation cases satisfied this stability criterion with the Transformer-predicted cost matrices.

The following section describes the dataset generation process.

III. DATASET GENERATION AND ANALYSIS

The DC motor model and the MPC formulation used as the basis for dataset generation and controller design are presented in what follows. Since collecting a large number of experimentally tuned MPC configurations is infeasible, the dataset was generated in simulation, using DC motor models identified from real measurements. In this way, the generated samples remain physically connected to the experimental system while allowing a sufficiently large and diverse set of operating cases to be produced.

The generation process starts from several fitted DC motor parameter sets instead of a single nominal model. Each model is described by electrical and mechanical parameters such as armature resistance, armature inductance, rotor inertia, viscous friction coefficient, torque constant, and back-EMF constant. These parameters are used to obtain the discrete-time matrices A_d , B_d , and E_d used in the MPC prediction model. Considering multiple identified models introduces plant variability and allows the dataset to reflect differences in motor dynamics observed in practice.

For each motor model, multiple operating scenarios are simulated. A scenario is defined by the reference trajectory, initial condition, disturbance profile, and actuator constraints. The reference signals include both step-like and time-varying speed commands, while the disturbance term models load variations acting on the motor shaft. Therefore, each generated sample captures both model-dependent uncertainty and scenario-dependent behavior.

The experimental measurements are not used directly as supervised targets. Instead, they are used to identify realistic motor models, after which the MPC tuning targets are obtained through offline simulation and optimization. This approach makes it possible to generate a larger dataset than would be feasible through manual tuning or repeated physical experiments.

For every generated scenario, the MPC stage-cost matrices are selected through an offline optimization procedure. The diagonal entries of the matrices are parameterized in logarithmic form as

$$Q = \text{diag}(10^{q_1}, 10^{q_2}), \quad R = \text{diag}(10^{r_1}), \quad (6)$$

where q_1 , q_2 , and r_1 are the optimization variables. This parameterization guarantees positive weights and allows the search procedure to explore values across several orders of magnitude.

Each candidate set of parameters is evaluated through closed-loop MPC simulation. The objective function accounts for tracking performance, control effort, response smoothness, undesirable transient behavior, and possible constraint violations. The optimization combines Latin Hypercube Sampling for global exploration with Powell local optimization for refinement. Although this procedure is computationally expensive, it is performed only once during dataset generation and is not part of the online adaptation stage.

After the optimized stage-cost matrices are obtained, the terminal cost-function parameter P is computed as the solution of the discrete-time algebraic Riccati equation (DARE),

$$P = Q + A_d^T P A_d - A_d^T P B_d (R + B_d^T P B_d)^{-1} B_d^T P A_d, \quad (7)$$

which yields the optimal infinite-horizon value function from the terminal state. Using P ensures that the finite-horizon MPC objective approximates the infinite-horizon optimal cost, providing a principled bound on the closed-loop performance beyond the prediction horizon.

Each sample is therefore associated with an MPC configuration adapted to a specific motor model and operating scenario.

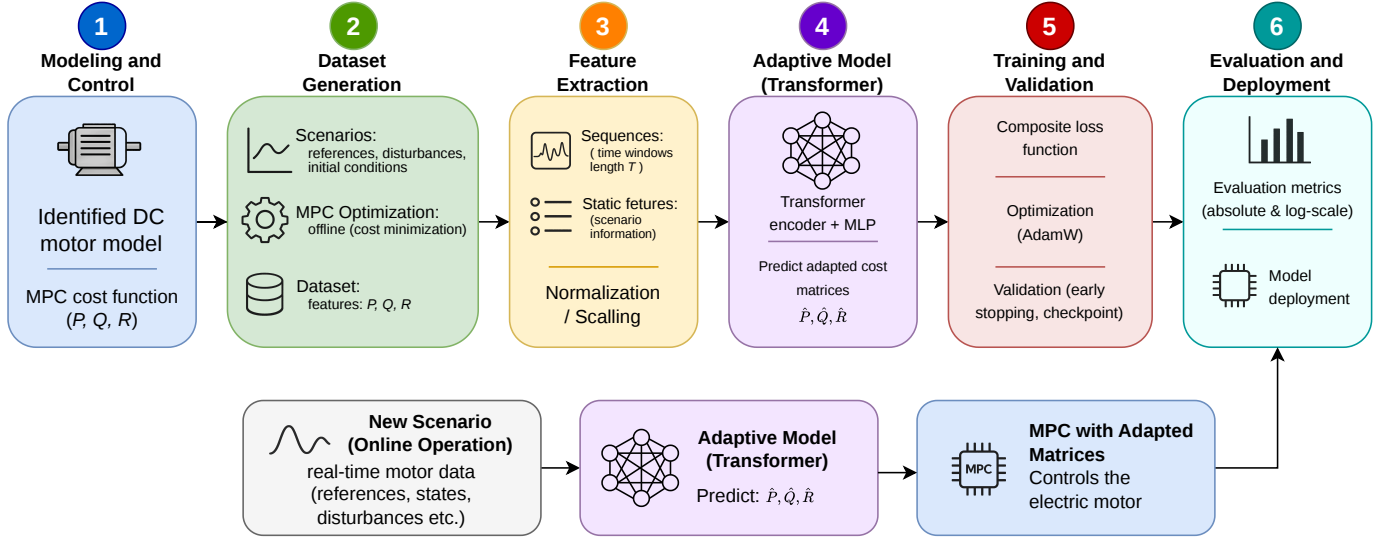


Fig. 1. Workflow of the proposed Transformer-based MPC cost-matrix adaptation framework for DC motor control, covering offline data generation and training followed by online inference.

To improve numerical conditioning and make the supervised learning problem more stable, the target values are represented as logarithms of the diagonal entries of the cost matrices:

$$\theta = \begin{bmatrix} \log_{10}(P_{11}) \\ \log_{10}(P_{22}) \\ \log_{10}(Q_{11}) \\ \log_{10}(Q_{22}) \\ \log_{10}(R_{11}) \end{bmatrix}. \quad (8)$$

The vector θ represents the supervised target used during training. It defines the MPC cost configuration that the Transformer model learns to infer from the motor parameters and operating conditions.

Several dataset versions were created during development. Initial smaller datasets were used to validate the generation pipeline and to inspect issues related to scaling, outliers, and unbalanced target distributions. Based on these analyses, a recalibration stage was introduced in which the optimization objective was refined while keeping the same family of identified motor models and simulated operating scenarios.

The final dataset used in the experimental evaluation contains 3000 recalibrated samples. The samples were randomly split into 90% training and 10% validation subsets, with no separate test set. Its analysis included the distribution of the logarithmic target parameters, the relative scale of the resulting cost matrices, and the presence of extreme operating cases. Statistical visualizations and target-space projections were used to detect dense regions, sparsely represented areas, and potential outliers. These diagnostics showed that the distribution and consistency of the optimized MPC configurations influence the performance of the learning model.

The recalibrated dataset produced a more balanced target space and a more consistent mapping between operating conditions and MPC cost parameters. As reflected in the experimental results, this improved the prediction accuracy

and generalization capability of the Transformer-based tuning model.

IV. TRANSFORMER-BASED COST MATRIX ADAPTATION FRAMEWORK

The framework is designed to learn the relationship between operating conditions and MPC cost-function parameters from a large set of automatically generated tuning examples. An overview of the complete workflow is presented in Fig. 1. The process starts from the discrete-time DC motor model and a set of simulated operating scenarios defined by reference trajectories, disturbance profiles, initial conditions, actuator constraints, motor parameters, and prediction horizon settings. For each scenario, an offline MPC tuning procedure is performed to obtain suitable cost-function parameters. The optimized configurations are then stored together with the corresponding sequential and static descriptors, forming the supervised dataset used for training.

After training, the model acts as an adaptation module for unseen operating conditions. Instead of repeating the computationally expensive offline optimization, it receives the descriptors of a new scenario and directly predicts the MPC cost configuration used by the predictive controller to compute the control action applied to the DC motor.

Each sample is represented using two types of inputs. The first is the sequential input $X_s \in \mathbb{R}^{T \times d_s}$, where T denotes the observation window length and d_s represents the number of temporal features at each time step. X_s captures the time evolution of the reference signal, motor response, previous control actions, and disturbance over the observation window. The second is the static input $x_c \in \mathbb{R}^{d_c}$, where d_c is the number of context features encoding scenario-level information that remains constant over time, including the identified motor parameters and quantities derived from the discrete-time prediction model.

The model output is the logarithmic representation of the MPC cost-function parameters. For the considered DC motor system, with $n = 2$ states and $m = 1$ control input, the predicted vector is

$$\hat{\theta} = [\hat{p}_1 \ \hat{p}_2 \ \hat{q}_1 \ \hat{q}_2 \ \hat{r}_1]^T. \quad (9)$$

The corresponding weighting matrices are reconstructed as

$$\begin{aligned} \hat{P} &= \text{diag}(10^{\hat{p}_1}, 10^{\hat{p}_2}), \quad \hat{Q} = \text{diag}(10^{\hat{q}_1}, 10^{\hat{q}_2}), \\ \hat{R} &= \text{diag}(10^{\hat{r}_1}). \end{aligned} \quad (10)$$

This formulation ensures positive diagonal weights and improves numerical conditioning by keeping the learning problem in the logarithmic domain.

The temporal input is processed using a Transformer encoder, whose self-attention mechanism captures dependencies across the observation window without recurrent connections. The sequence X_s is first projected into a latent space of dimension d_m through a linear embedding layer as $Z_0 = X_s W_e + b_e$, where d_m is the model dimension, $W_e \in \mathbb{R}^{d_s \times d_m}$ is the learnable projection matrix, and $b_e \in \mathbb{R}^{d_m}$ is a learnable bias vector that allows the projection to shift the representation independently of the input. Since the Transformer has no inherent notion of temporal order, a positional embedding is added to encode the position of each step within the sequence: $Z = Z_0 + E_{pos}$, where $E_{pos} \in \mathbb{R}^{T \times d_m}$ is a matrix that assigns a unique time-dependent signal to each position, enabling the model to distinguish between earlier and later observations.

The embedded sequence H is then passed through the encoder,

$$H = \text{TransformerEncoder}(Z) = [h_1, h_2, \dots, h_t], \quad (11)$$

where $H \in \mathbb{R}^{T \times d_m}$ contains the encoded temporal features and $h_t \in \mathbb{R}^{d_m}$ is the latent vector at time step t , a compressed internal encoding of the input learned by the encoder. The implemented model uses two Transformer encoder layers, four self-attention heads, and a latent dimension of d_m . Dropout regularization, which randomly deactivates a fraction of activations during training to prevent overfitting, is applied within the encoder.

The unscaled attention score at each time step is computed as

$$a_t = w_a^T \tanh(W_a h_t + b_a), \quad (12)$$

and normalized across the temporal dimension using the softmax function:

$$\alpha_t = \frac{\exp(a_t)}{\sum_{\tau=1}^T \exp(a_\tau)}. \quad (13)$$

Here, the softmax operation is used only to obtain normalized attention weights over the input sequence, not to perform classification. The overall task remains a supervised regression problem, since the network predicts continuous logarithmic MPC cost parameters.

The sequence representation is obtained as

$$h_s = \sum_{t=1}^T \alpha_t h_t.$$

The static feature vector is processed separately by a multi-layer perceptron and then fused with h_s . The resulting representation is passed through a regression module that outputs the adapted MPC cost parameters for the current operating scenario.

Training is formulated as supervised regression, using the logarithmic MPC parameters obtained from offline optimization as targets. Let θ_P , θ_Q , and θ_R denote the target components associated with the terminal, state, and input weighting matrices, and let $\hat{\theta}_P$, $\hat{\theta}_Q$, and $\hat{\theta}_R$ be the corresponding predictions. The total loss is

$$L = \lambda_P L_P + \lambda_Q L_Q + \lambda_R L_R + \lambda_u L_u, \quad (14)$$

where λ_P , λ_Q , λ_R , and λ_u are weighting coefficients.

The final model uses Huber loss for the terminal and state-weighting parameters,

$$L_P = \text{Huber}(\hat{\theta}_P, \theta_P), \quad L_Q = \text{Huber}(\hat{\theta}_Q, \theta_Q), \quad (15)$$

and mean squared error for the input-weighting parameter,

$$L_R = \|\hat{\theta}_R - \theta_R\|_2^2. \quad (16)$$

An additional auxiliary term penalizes the prediction error of the control input associated with the generated training sample,

$$L_u = \|\hat{u} - u\|_2^2. \quad (17)$$

This mixed-loss formulation was selected empirically because it improved robustness to outliers while maintaining stable convergence. All input features are normalized using statistics computed only from the training set. The network is optimized with AdamW, weight decay, dropout regularization, and a cosine learning-rate schedule with warm-up. The final model is selected according to the lowest validation loss obtained during training.

V. EXPERIMENTAL EVALUATION

This section evaluates the Transformer-based model for MPC cost matrix adaptation. The evaluation follows two stages. First, the generated datasets are analyzed to verify the quality and consistency of the offline-optimized MPC cost parameters used as supervised learning targets. Second, several model and training configurations are compared, and the selected model is evaluated in a closed-loop vs. the offline-optimized MPC controller.

A. Dataset Analysis and Final Configuration

Before training the Transformer-based adaptation model, the generated datasets were analyzed to verify the quality and consistency of the supervised targets. This step is important because the model learns the mapping between motor and scenario descriptors and the MPC cost parameters obtained through offline optimization. Therefore, an unbalanced or inconsistent target distribution can directly affect prediction accuracy and generalization.

The dataset generation process was based on DC motor models identified from experimental measurements. Table I reports one representative fitted parameter set used to construct

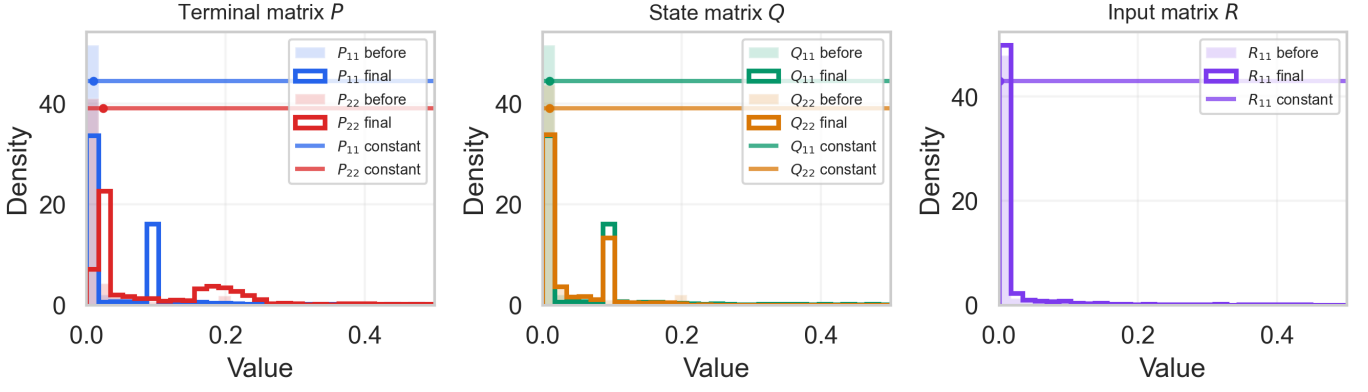


Fig. 2. Distributions of the MPC cost parameters, with fixed reference values indicated by horizontal markers.

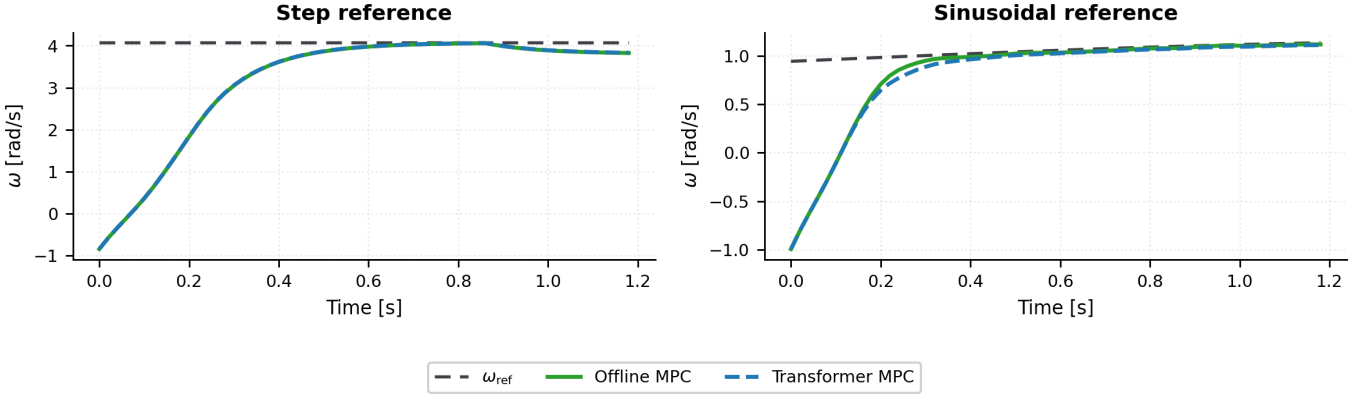


Fig. 3. Closed-loop angular velocity tracking for representative validation scenarios using Transformer-predicted and offline-optimized MPC cost matrices.

TABLE I
IDENTIFIED DC MOTOR PARAMETERS

Symbol	Description	Value	Unit
R_a	Armature resistance	0.0787	Ω
L_a	Armature inductance	4.83×10^{-5}	H
J	Moment of inertia	4.67×10^{-4}	$\text{kg}\cdot\text{m}^2$
b	Viscous friction coefficient	9.61×10^{-4}	$\text{N}\cdot\text{m}\cdot\text{s}/\text{rad}$
K_t	Torque constant	0.0159	$\text{N}\cdot\text{m}/\text{A}$
K_e	Back-EMF constant	0.0159	$\text{V}\cdot\text{s}/\text{rad}$

the discrete-time prediction model for the MPC simulations. These parameters describe the electrical and mechanical dynamics of the motor and provide the physical basis for the generated operating scenarios.

Several dataset versions were produced during development. The initial versions were used to validate the simulation and offline optimization pipeline, but their analysis revealed uneven target coverage, extreme optimized weights, and inconsistent scaling between the entries of the MPC cost matrices. Since the parameters of P , Q , and R span different numerical ranges, the target values were also inspected carefully before selecting the final training set.

Based on this preliminary analysis, a recalibration stage was introduced. The same family of identified motor models, reference trajectories, disturbances, and operating scenarios was preserved, while the offline optimization objective used to select the MPC weights was refined. For each scenario,

the offline optimization produced scenario-specific reference matrices P^* , Q^* , and R^* . Their diagonal entries, p_1^* , p_2^* , q_1^* , q_2^* , and r_1^* , were stored as supervised learning targets for the Transformer model.

A fixed non-optimized MPC configuration was also included as a reference baseline. This configuration used constant state and input weights,

$$Q_{\text{const}} = \begin{bmatrix} 0.01 & 0 \\ 0 & 0.01 \end{bmatrix}, \quad R_{\text{const}} = [0.001],$$

with the corresponding representative terminal matrix

$$P_{\text{const}} \approx \begin{bmatrix} 0.0100 & 0 \\ 0 & 0.0242 \end{bmatrix}.$$

This baseline was not obtained through optimization and is used only to show where a manually selected constant tuning lies relative to the optimized dataset targets.

Fig. 2 compares the distributions of the optimized MPC cost parameters for the analyzed dataset configurations. The parameters are grouped according to the corresponding MPC cost matrices, with P_{11} and P_{22} shown together, Q_{11} and Q_{22} shown together, and R_{11} shown separately. Each diagonal parameter is represented with a distinct color, while the preliminary and final dataset configurations are distinguished by the histogram style. The horizontal colored markers indicate the fixed non-optimized constant values used for comparison.

TABLE II
COMPARISON OF TESTED MODEL AND LOSS CONFIGURATIONS

Configuration	Samples	R_P^2	R_Q^2	R_R^2	Mean log-MAE
PatchTST log-diagonal encoder	1000	0.545	0.563	0.200	1.165
Attention + Huber	1000	0.679	0.629	0.175	0.958
Attention + MSE	1000	0.596	0.591	0.265	1.060
Attention + Huber	3000	0.594	0.575	0.306	0.330
Attention + Huber (P, Q) + MSE(R)	3000	0.574	0.577	0.310	0.344

TABLE III
FINAL MODEL PREDICTION PERFORMANCE

Matrix	$R_{\log}^2$	Log-MAE	Median Rel. Error
P	0.574	0.322	24.9%
Q	0.577	0.335	29.1%
R	0.310	0.373	29.2%

The comparison shows that the recalibration stage reduced the dominance of unsuitable preliminary configurations and produced a more consistent supervised target set. The final dataset contains 3000 samples and was selected for the experimental evaluation because it combines physically consistent generated scenarios, improved target distributions, and more stable learning behavior during training.

B. Model Selection and Closed-Loop Evaluation

Several model and training configurations were evaluated on the final dataset in order to select the Transformer estimator used in the closed-loop experiments. The tested configurations included a patch-based temporal encoder, attention pooling with Huber loss, attention pooling with MSE loss, and a mixed loss using Huber loss for P and Q and MSE loss for R .

Table II summarizes the validation results. The reported R^2 values are computed in the logarithmic diagonal space for P , Q , and R , while the mean log-MAE is the average absolute error across the predicted logarithmic diagonal entries. Higher R^2 values and lower log-MAE values indicate better prediction quality.

The attention-based Transformer variants outperformed the PatchTST-style encoder in the tested configurations. Increasing the dataset size to the final 3000-sample recalibrated configuration substantially reduced the logarithmic prediction error. The mixed Huber (P, Q) + MSE (R) formulation was selected for the final evaluation because it provided a balanced compromise between the prediction of the terminal and state weighting matrices and the more difficult input weighting matrix R .

Table III reports the prediction performance of the selected model for each MPC cost matrix. This separate evaluation is included to show how the final configuration behaves for the terminal, state, and input weighting matrices individually.

The results show that the terminal matrix P and the state matrix Q are predicted with similar accuracy in the logarithmic target space. The input matrix R obtains a lower $R_{\log}^2$ value, indicating that the input penalty is more difficult to estimate. This behavior is consistent with the larger variability of the optimized input-weighting values and motivates the use of a separate MSE term for R in the final mixed-loss formulation. Since R controls the penalty applied to the control input, prediction errors directly influence the balance between tracking performance and control effort. Underestimating R can lead to more aggressive voltage commands, higher control energy,

TABLE IV
CLOSED-LOOP TRACKING PERFORMANCE

Metric	Offline MPC	Transformer MPC
IAE	0.907	0.234
ISE	0.496	0.060
Steady-state error	0.526	0.125
Settling time [s]	2.400	2.400
Overshoot	0.842	0.336

greater input variation, and an increased risk of saturation or overshoot. In contrast, overestimating R results in smoother but more conservative control actions, which may increase the tracking and settling errors.

The selected model was then evaluated in closed-loop by comparing two MPC controllers on the same validation scenarios. The first controller uses the offline-optimized matrices P^* , Q^* , and R^* , obtained during dataset generation and used as supervised targets. The second controller uses the Transformer-predicted matrices $\hat{P}$, $\hat{Q}$, and $\hat{R}$, computed directly from the motor parameters and scenario descriptors.

For each validation scenario, the offline-optimized matrices are scenario-dependent and have the diagonal form

$$P^* = \begin{bmatrix} p_1^* & 0 \\ 0 & p_2^* \end{bmatrix}, \quad Q^* = \begin{bmatrix} q_1^* & 0 \\ 0 & q_2^* \end{bmatrix}, \quad R^* = [r_1^*].$$

These matrices represent the reference MPC configuration used for comparison with the Transformer-predicted matrices.

For each validation scenario, both controllers used the same discrete-time motor model, prediction horizon, constraints, reference trajectory, disturbance profile, and initial condition. Thus, the comparison isolates the effect of replacing the offline-optimized MPC cost matrices with Transformer-predicted matrices.

Fig. 3 shows representative closed-loop angular velocity responses for a step reference and a sinusoidal reference. The dashed black curve denotes the reference trajectory, the green curve corresponds to the MPC controller using offline-optimized cost matrices, and the dashed blue curve corresponds to the MPC controller using Transformer-predicted cost matrices.

The responses show that the Transformer-predicted matrices lead to tracking behavior close to that obtained with the offline-optimized MPC controller. The two controllers exhibit similar transient and steady-state behavior in the selected validation scenarios, indicating that the learned mapping can provide usable MPC cost configurations without repeating the offline optimization procedure at test time.

Table IV summarizes the quantitative closed-loop tracking performance for the evaluated validation scenarios. The metrics include integral error measures, final tracking error, settling time, and overshoot, allowing the trajectory comparison in Fig. 3 to be assessed numerically.

The values in Table IV show that, for the evaluated scenarios, the Transformer-adapted MPC achieves lower IAE, ISE, steady-state error, and overshoot than the offline-tuned MPC, while preserving the same settling time. This indicates that the learned adaptation mechanism is able to generate weighting

terms that lead to effective closed-loop behavior and can be deployed without repeating the offline tuning procedure.

At the same time, these results should be interpreted in relation to the role of the offline-tuned controller in the proposed framework. Since the offline optimization is used to generate the supervised learning targets, it serves as the tuning reference rather than as a fully independent competing controller. The lower values obtained by the Transformer-adapted MPC for some time-domain metrics suggest that the learned model does not merely reproduce the target parameters, but may also smooth or regularize the resulting controller configuration across operating scenarios. In addition, the offline tuning objective and the evaluation metrics reported in Table IV are not identical, which can explain why improved IAE, ISE, or overshoot may be observed even when learning from offline-optimized targets.

Overall, the experimental results show that the Transformer-based model learns a meaningful mapping between motor and scenario descriptors and the optimized MPC weighting terms P , Q , and R . Although the input-weighting term remains more difficult to predict than the state-related terms, the closed-loop evaluation indicates that the predicted weighting terms are sufficiently accurate to preserve the main tracking behavior of the offline-tuned MPC controller and, in the evaluated cases, can improve several time-domain performance indicators.

VI. CONCLUSIONS

This paper presented a Transformer-based framework for adapting MPC cost matrices in DC motor control. The proposed approach learns the mapping between motor and scenario descriptors and suitable MPC cost-function parameters, using a supervised dataset generated from DC motor models fitted on real measurements, simulated operating conditions, and offline MPC optimization.

The final model combines a Transformer encoder, attention pooling, static-feature processing, and a regression module that predicts the logarithmic representation of the MPC weighting matrices. Experimental evaluation on the recalibrated 3000-sample dataset showed that the terminal and state-weighting matrices can be predicted with positive performance in both logarithmic and absolute domains. The input-weighting matrix remains more difficult to estimate, mainly due to its wider dynamic range, but the model still captures its order of magnitude in the logarithmic domain.

Closed-loop validation on representative scenarios showed that the Transformer-adapted cost matrices produce stable responses and tracking behavior close to the offline-optimized configurations. These results support the feasibility of using Transformer-based models as a data-driven mechanism for MPC cost adaptation, reducing the dependence on manual tuning and repeated offline optimization.

Future work will focus on expanding the dataset with a broader range of operating conditions, improving the prediction of the input-weighting matrix, conducting a broader quantitative closed-loop evaluation, and validating the adapted

controller more extensively on experimental hardware and other control applications.

REFERENCES

- [1] M. Schwenzer, M. Ay, T. Bergs, and D. Abel, "Review on Model Predictive Control: An Engineering Perspective," *The International Journal of Advanced Manufacturing Technology*, vol. 117, no. 5-6, pp. 1327–1349, 2021.
- [2] S. Vazquez, J. I. Leon, L. G. Franquelo, J. Rodriguez, H. A. Young, A. Marquez, and P. Zanchetta, "Model Predictive Control: A Review of Its Applications in Power Electronics," *IEEE Industrial Electronics Magazine*, vol. 8, no. 1, pp. 16–31, 2014.
- [3] W. Edwards, G. Tang, G. Mamakoukas, T. Murphey, and K. Hauser, "Automatic Tuning for Data-driven Model Predictive Control," in *IEEE International Conference on Robotics and Automation*, Xi'an, China, 2021, pp. 7379–7385.
- [4] P. Verheijen, V. Breschi, and M. Lazar, "Handbook of linear data-driven predictive control: Theory, implementation and design," *Annual Reviews in Control*, vol. 56, p. 100914, 2023.
- [5] L. Zhang, L. Xie, H. Su, and J. Liu, "Data-driven auto-tuning strategy for RTO-MPC based on Bayesian optimization," *Computers & Chemical Engineering*, vol. 187, p. 108743, 2024.
- [6] L. Hewing, K. Wabersich, M. Menner, and M. Zeilinger, "Learning-Based Model Predictive Control: Toward Safe Learning in Control," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 3, pp. 269–296, 2020.
- [7] A. Maxim, O. Pauca, and C. F. Caruntu, "Coalitional Distributed Model Predictive Control Strategy with Switching Topologies for Multi-Agent Systems," *Electronics*, vol. 13, no. 4, p. 792, 2024.
- [8] M. M. Morato and M. S. Felix, "Data Science and Model Predictive Control: A survey of recent advances on data-driven MPC algorithms," *Journal of Process Control*, vol. 144, p. 103327, 2024.
- [9] C. G. Arango, H. Asadi, L. Kooijman, and C. P. Lim, "Neural Networks for Fast Optimisation in Model Predictive Control: A Review," *arXiv preprint arXiv:2309.02668*, 2023.
- [10] S. Hirt, M. Pfeifferkorn, and R. Findeisen, "Safe and Stable Closed-Loop Learning for Neural-Network-Supported Model Predictive Control," in *63rd IEEE Conference on Decision and Control*, Milan, Italy, 2024, pp. 4764–4770.
- [11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [12] Q. Wen, T. Zhou, C. Zhang, W. Chen, Z. Ma, J. Yan, and L. Sun, "Transformers in Time Series: A Survey," in *32nd International Joint Conference on Artificial Intelligence*, Macao, China, 2023, pp. 6778–6786.
- [13] V. Zinage, A. Khalil, and E. Bakolas, "TransformerMPC: Accelerating Model Predictive Control via Transformers," in *IEEE International Conference on Robotics and Automation*, Atlanta, GA, USA, 2025, pp. 9221–9227.
- [14] D. Celestini, D. Gammelli, T. Guffanti, S. D'Amico, E. Capello, and M. Pavone, "Transformer-Based Model Predictive Control: Trajectory Optimization via Sequence Modeling," *IEEE Robotics and Automation Letters*, vol. 9, no. 11, pp. 9820–9827, 2024.
- [15] S. Wu, J. Wu, X. Cao, F. Zhang, G. Yu, J. Zhao, Y. Qu, F. Ma, and J. Duan, "Transformer-based explicit model predictive control with variable prediction horizon," *Engineering Applications of Artificial Intelligence*, vol. 172, p. 114345, 2026.
- [16] M. Ruderman, J. Krettek, F. Hoffmann, and T. Bertram, "Optimal State Space Control of DC Motor," *IFAC Proceedings Volumes*, vol. 41, no. 2, pp. 5796–5801, 2008.
- [17] M. R. Stankovic, S. M. Manojlovic, and M. B. Naumovic, "Identification of the parameters of a DC motor state space model," *Vojnotehnički glasnik*, vol. 61, no. 2, pp. 23–36, 2013.
- [18] A. A. Masoud, M. Abu-Ali, and A. Al-Shaikhi, "Experimental Determination of an Extended DC Servo-Motor State Space Model: An Undergraduate Experiment," *IEEE Access*, vol. 8, pp. 4908–4923, 2020.
- [19] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed. Madison, WI, USA: Nob Hill Publishing, 2017.